

দিন 5

Recursion, Memoization, DP ও Greedy

বড় সমস্যাকে ছোট সমস্যায় ভাঙা — সবচেয়ে "ভয়ের" টপিক, কিন্তু আসলে একটা প্যাটার্ন

আজ যা শিখবো

- Recursion: base case, recursive case, call stack
- Recursion tree আর কেন সাধারণ Fibonacci ধীর
- Memoization (Top-down DP)
- Tabulation (Bottom-up DP) — DP সমাধানের ৪ ধাপ
- Greedy: কখন কাজ করে, কখন করে না
- ৬টি গুরুত্বপূর্ণ প্রশ্ন

কোর্স ভিডিও: ২৪-২৫, ২৯-৩২

দিন শেষে তুমি পারবে

- যেকোনো recursive ফাংশনের base case আর recursive case চিনতে পারবে
- Fibonacci-কে $O(2^n)$ থেকে $O(n)$ -এ নামানো ব্যাখ্যা করতে পারবে
- Climbing Stairs, House Robber নিজে লিখতে পারবে
- Greedy কেন Coin Change-এ ব্যর্থ হয় উদাহরণসহ বলতে পারবে

স্ট্যান্ডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

১ ঘণ্টা	Recursion কনসেপ্ট + factorial/sum/power
১ ঘণ্টা	Fibonacci — ৪টা ভার্শন (সবচেয়ে গুরুত্বপূর্ণ)
১ ঘণ্টা	Climbing Stairs + House Robber
৪৫ মিনিট	Coin Change (Greedy বনাম DP)
৪৫ মিনিট	Merge Intervals (Greedy)
৩০ মিনিট	খিওরি + চিট শিট

অধ্যায় ১: Recursion — ফাংশন যখন নিজেকে কল করে

Recursion মানে একটা ফাংশন নিজের ভেতর থেকে নিজেকেই কল করে, প্রতিবার একটু ছোট সমস্যা দিয়ে — যতক্ষণ না সমস্যাটা এত ছোট হয় যে উত্তর সরাসরি জানা।

বাস্তব উদাহরণ: সিনেমা হলে সারি নম্বর

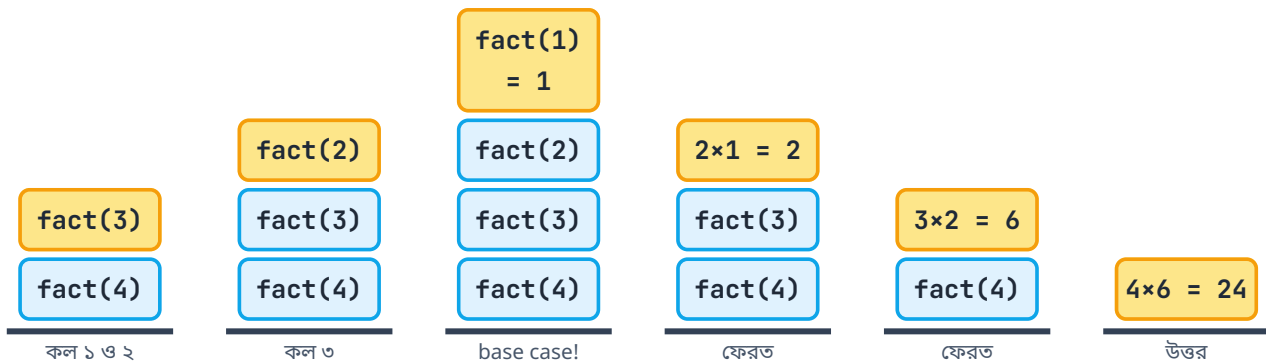
অন্ধকার হলে তুমি জানতে চাও কোন সারিতে বসেছো। সামনের জনকে জিজ্ঞেস করো "তুমি কোন সারিতে?" সে-ও জানে না, তার সামনের জনকে জিজ্ঞেস করে... একদম প্রথম সারির লোক বলে "১" (base case)। তারপর উত্তর পেছনে ফিরতে থাকে: ২, ৩, ৪... প্রত্যেকে সামনের উত্তরের সাথে ১ যোগ করে।

প্রতিটা recursion-এ দুইটা অংশ থাকতেই হবে

অংশ	কাজ	না থাকলে কী হবে
Base case	সবচেয়ে ছোট সমস্যার সরাসরি উত্তর — এখানে থামো	অসীম কল → Stack Overflow
Recursive case	সমস্যাকে ছোট করে নিজেকে কল করো	কখনো base case-এ পৌঁছাবে না

Call Stack — ভেতরে কী হয়?

প্রতিটা ফাংশন কল একটা stack-এ জমা হয় (দিন ৪-এর Stack!)। base case-এ পৌঁছালে উত্তর নিয়ে একে একে pop হয়।



Recursion কখন দরকার?

- সমস্যাটা একই ধরনের ছোট সমস্যায় ভাঙা যায় (factorial, fibonacci)
- Tree আর Graph traversal (দিন ৬)
- Divide & Conquer (merge sort, quick sort)
- সব সম্ভাবনা খোঁজা — subsets, permutations (backtracking)

সাধারণ লুপ দিয়ে সহজে হলে লুপই ভালো — recursion-এ call stack-এর বাড়তি মেমোরি লাগে।

প্রশ্ন 5.1

Recursion Basics: Factorial, Sum, Power

Recursion দিয়ে তিনটা ছোট সমস্যা

Easy

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Recursion-এ হাত পাকানোর জন্য। ইন্টারভিউতে ওয়ার্ম-আপ হিসেবে "factorial recursive লেখো" আসে।

সমস্যাটা কী?

Recursion দিয়ে লেখো: (ক) $n! = n \times (n-1) \times \dots \times 1$, (খ) array-র সব সংখ্যার যোগফল, (গ) x^n (দ্রুত পদ্ধতিতে)।

উদাহরণ

`factorial(5) = 120``sumArray([1, 2, 3, 4]) = 10``power(2, 10) = 1024`

(ক) Factorial

1. Base: $n \leq 1$ হলে 1
2. Recursive: $n \times \text{factorial}(n - 1)$

PHP

```
function factorial(int $n): int {
    if ($n <= 1) return 1;           // base case
    return $n * factorial($n - 1);   // recursive case
}
echo factorial(5); // 120
```

Time: $O(n)$ Space: $O(n)$ – call stack

(খ) Array-র যোগফল

1. Base: শেষ index পার হলে 0
2. Recursive: `arr[i] + বাকি অংশের যোগফল`

PHP

```
function sumArray(array $arr, int $i = 0): int {
    if ($i >= count($arr)) return 0; // base case
    return $arr[$i] + sumArray($arr, $i + 1); // এই element + বাকিদের যোগ
}
echo sumArray([1, 2, 3, 4]); // 10
```

Time: $O(n)$ Space: $O(n)$ **(গ) Power — সাধারণ বনাম দ্রুত (Divide & Conquer)**

1. সাধারণ: $x \times \text{power}(x, n-1) \rightarrow n$ টা কল, $O(n)$
2. দ্রুত: $x^n = (x^{n/2})^2$. অর্ধেকের উত্তর একবার বের করে নিজের সাথে গুণ করো $\rightarrow O(\log n)$
3. n বিজোড় হলে আরেকবার x দিয়ে গুণ

PHP

```
function powerSlow(int $x, int $n): int {
    if ($n === 0) return 1;
    return $x * powerSlow($x, $n - 1); //  $O(n)$ 
}

function powerFast(int $x, int $n): int {
    if ($n === 0) return 1;
    $half = powerFast($x, intval($n / 2)); // একবারই কল!
    return ($n % 2 === 0) ? $half * $half : $half * $half * $x;
}

echo powerSlow(2, 10) . " " . powerFast(2, 10); // 1024 1024
```

Time: $O(\log n)$ Space: $O(\log n)$

ভুল করো না: `powerFast($x, $n/2) * powerFast($x, $n/2)` লিখলে দুইবার কল হয়, আবার $O(n)$ হয়ে যায়।

এজ কেস (এগুলো ভুলবে না)

- $\text{factorial}(0) = 1$ (গাণিতিক নিয়ম)
- নেগেটিভ n — প্রশ্ন করো কী করতে হবে
- বড় n -এ `PHP_INT_MAX` পার হয়ে float হয়ে যায় (20! পর্যন্ত int-এ ধরে)

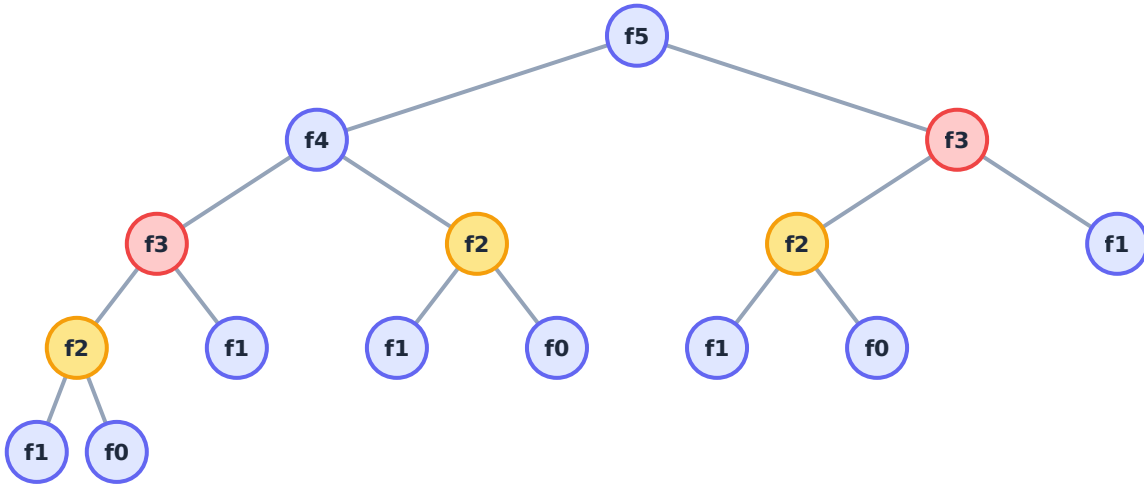
ইন্টারভিউতে যা বলবে

- প্রতিটা recursive ফাংশন লেখার আগে মুখে বলো: "আমার base case হলো ..., আর প্রতিবার সমস্যা ছোট হচ্ছে কারণ ..."

অধ্যায় ২: Memoization ও Dynamic Programming

সমস্যা: একই কাজ বারবার

সাধারণ recursive Fibonacci-তে `fib(5)` বের করতে গাছটা দেখো — **fib(3)** ২ বার, **fib(2)** ৩ বার হিসাব হচ্ছে! n বাড়লে এই পুনরাবৃত্তি বিস্তারিত হয় $\rightarrow O(2^n)$ । `fib(50)` হিসাব করতে কয়েক মিনিট লাগে।



লাল = `fib(3)` দুইবার, হলুদ = `fib(2)` তিনবার — একই হিসাব বারবার।

সমাধান ১: Memoization (Top-down)

একটা খাতা (array/map) রাখো। কোনো উত্তর একবার হিসাব করলে খাতায় লিখে রাখো। পরেরবার একই প্রশ্ন এলে হিসাব না করে খাতা থেকে দাও। Recursion একই থাকে, শুধু শুরুতে "খাতায় আছে কি?" চেক যোগ হয়।

বাস্তব উদাহরণ

কেউ জিজ্ঞেস করলো ১৭×২৩ কত? তুমি হিসাব করে ৩৯১ বের করলে আর কাগজে লিখে রাখলে। একই প্রশ্ন আবার এলে সরাসরি কাগজ দেখে বলো — আবার হিসাব করো না।

সমাধান ২: Tabulation (Bottom-up)

Recursion বাদ দাও। সবচেয়ে ছোট সমস্যা থেকে শুরু করে একটা টেবিল (array) ভরতে ভরতে বড় সমস্যায় যাও। কোনো call stack লাগে না।

Dynamic Programming কখন ব্যবহার করবে?

দুইটা বৈশিষ্ট্য থাকলে সমস্যাটা DP দিয়ে সমাধান করা যায়:

1. **Overlapping subproblems:** একই ছোট সমস্যা বারবার আসে (`fib(3)` দুইবার)
2. **Optimal substructure:** বড় সমস্যার সেরা উত্তর ছোট সমস্যাগুলোর সেরা উত্তর দিয়ে তৈরি

DP সমাধানের ৪ ধাপ (প্রতিটা DP প্রশ্নে এটা করো)

1. **State:** $dp[i]$ মানে কী? (যেমন: "i-তম সিঁড়িতে পৌঁছানোর উপায়ের সংখ্যা")
2. **Transition:** $dp[i]$ কীভাবে আগের state থেকে আসে? ($dp[i] = dp[i-1] + dp[i-2]$)
3. **Base case:** সবচেয়ে ছোট মানগুলো ($dp[0]$, $dp[1]$)
4. **Answer:** কোন ঘরে উত্তর? ($dp[n]$)

	Memoization (Top-down)	Tabulation (Bottom-up)
দিক	বড় থেকে ছোট (recursion)	ছোট থেকে বড় (loop)
লেখা	সহজ — recursion + cache	একটু চিন্তা করতে হয় ক্রম নিয়ে
Stack overflow ঝুঁকি	আছে (গভীর recursion)	নেই
শুধু দরকারি state হিসাব	হ্যাঁ	না, সব state হিসাব হয়

প্রশ্ন 5.2

Fibonacci Number

৪টা ভাঙ্গনে — Recursion থেকে $O(1)$ space

Easy

LeetCode 509

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

DP শেখার প্রথম ধাপ। ইন্টারভিউতে প্রায়ই "recursive লেখো → এটা ধীর কেন? → ঠিক করো" — এই ধারায় প্রশ্ন আসে।

সমস্যাটা কী?

$F(0) = 0$, $F(1) = 1$, আর $F(n) = F(n-1) + F(n-2)$ । n দেওয়া থাকলে $F(n)$ বের করো।

উদাহরণ

Input: $n = 6$

Output: 8 // 0, 1, 1, 2, 3, 5, 8

Input: $n = 10$

Output: 55

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

প্রতিটা সংখ্যা তার আগের দুইটার যোগফল। খরগোশের বংশবৃদ্ধি, সূর্যমুখী ফুলের বীজের বিন্যাস — প্রকৃতিতে এই ধারা পাওয়া যায়।

ভাঙ্গন ১: সাধারণ Recursion

সংজ্ঞাটা সরাসরি কোডে লেখা। কিন্তু প্রতিটা কল দুইটা কল করে → কলের সংখ্যা প্রায় দ্বিগুণ হয় প্রতি লেভেলে।

PHP

```
function fibRec(int $n): int {
    if ($n <= 1) return $n; // base: F(0)=0, F(1)=1
    return fibRec($n - 1) + fibRec($n - 2);
}
echo fibRec(10); // 55 (fibRec(40) চালালে অনেক সময় লাগবে!)
```

Time: $O(2^n)$ Space: $O(n)$ — call stack-এর গভীরতা

ভাঙ্গন ২: Memoization (Top-down DP)

1. একটা \$memo array রাখো
2. হিসাব করার আগে দেখো memo-তে আছে কি না
3. হিসাব শেষে memo-তে রাখো

PHP

```
function fibMemo(int $n, array &$memo = []): int {
    if ($n <= 1) return $n;
    if (isset($memo[$n])) return $memo[$n];          // আগে হিসাব করা? সরাসরি দাও
    $memo[$n] = fibMemo($n - 1, $memo) + fibMemo($n - 2, $memo);
    return $memo[$n];
}
echo fibMemo(50);    // 12586269025 — মুহুর্তেই!
```

Time: $O(n)$ Space: $O(n)$

প্রতিটা n মাত্র একবার হিসাব হয়। `&$memo` (reference) না দিলে প্রতিটা কল আলাদা কপি পারে — memo কাজ করবে না!

ভার্সন ৩: Tabulation (Bottom-up DP)

1. $dp[0] = 0, dp[1] = 1$
2. $i = 2$ থেকে n : $dp[i] = dp[i-1] + dp[i-2]$

PHP

```
function fibTab(int $n): int {
    if ($n <= 1) return $n;
    $dp = [0, 1];
    for ($i = 2; $i <= $n; $i++) {
        $dp[$i] = $dp[$i - 1] + $dp[$i - 2];
    }
    return $dp[$n];
}
echo fibTab(50);    // 12586269025
```

Time: $O(n)$ Space: $O(n)$

ভার্সন ৪: Space Optimized — শুধু দুইটা ভেরিয়েবল

$dp[i]$ শুধু আগের দুইটা মানের উপর নির্ভর করে। পুরো array রাখার দরকার নেই — দুইটা ভেরিয়েবলই যথেষ্ট।

PHP

```
function fib(int $n): int {
    if ($n <= 1) return $n;
    $prev2 = 0; $prev1 = 1;
    for ($i = 2; $i <= $n; $i++) {
        $curr = $prev1 + $prev2;
        $prev2 = $prev1;
        $prev1 = $curr;
    }
    return $prev1;
}
echo fib(6) . " " . fib(50);    // 8 12586269025
```

Time: $O(n)$ Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

Tabulation: dp টেবিল বাম থেকে ডানে ভরা হচ্ছে। নীল = base case, সবুজ = উত্তর dp[6]



ধাপ 1 $dp[2] = dp[1] + dp[0] = 1 + 0 = 1$



ধাপ 2 $dp[3] = dp[2] + dp[1] = 1 + 1 = 2 \dots$ এভাবে প্রতিটা ঘর আগের দুইটা থেকে



ধাপ 3 $dp[6] = dp[5] + dp[4] = 5 + 3 = 8$



এজ কেস (এগুলো ভুলবে না)

- $n = 0 \rightarrow 0, n = 1 \rightarrow 1$
- বড় n -এ ($n > 92$) PHP int overflow হয়ে float — প্রশ্ন করো কত বড় n আসবে

ইন্টারভিউতে যা বলবে

- ইন্টারভিউতে এই ধারায় বলো: "recursive $O(2^n)$ কারণ একই subproblem বারবার $\rightarrow$ memoization দিয়ে $O(n) \rightarrow$ bottom-up দিয়ে stack এড়াই $\rightarrow$ শুধু দুইটা ভেরিয়েবল দিয়ে $O(1)$ space!"

প্রশ্ন 5.3

Climbing Stairs

সিঁড়ি বেয়ে ওঠার কয়টা উপায়

Easy

LeetCode 70

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Fibonacci-র ছদ্মবেশ! প্রথম "আসল" DP প্রশ্ন হিসেবে সবচেয়ে বেশি জিজ্ঞেস করা হয়। State আর transition চিন্তা করা শেখায়।

সমস্যাটা কী?

n ধাপের সিঁড়ি। প্রতিবার ১ বা ২ ধাপ উঠতে পারো। উপরে পৌঁছানোর মোট কয়টা আলাদা উপায় আছে?

উদাহরণ

Input: $n = 2$
 Output: 2 // (1+1), (2)
 Input: $n = 3$
 Output: 3 // (1+1+1), (1+2), (2+1)
 Input: $n = 5$
 Output: 8

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

৫ নম্বর ধাপে দাঁড়ানোর আগে তুমি শেষ পা দিয়েছো হয় ৪ নম্বর থেকে (১ ধাপ), নয়তো ৩ নম্বর থেকে (২ ধাপ)। তাই ৫-এ পৌঁছানোর উপায় = ৪-এ পৌঁছানোর উপায় + ৩-এ পৌঁছানোর উপায়। এটাই Fibonacci!

পদ্ধতি ১: সাধারণ Recursion

$ways(n) = ways(n-1) + ways(n-2)$ | Base: $ways(1) = 1$, $ways(2) = 2$

PHP

```
function climbRec(int $n): int {
    if ($n <= 2) return $n; // ways(1)=1, ways(2)=2
    return climbRec($n - 1) + climbRec($n - 2);
}
echo climbRec(5); // 8
```

Time: $O(2^n)$ Space: $O(n)$

পদ্ধতি ২: DP (Space Optimized)

1. **State:** $dp[i]$ = i নম্বর ধাপে পৌঁছানোর উপায়
2. **Transition:** $dp[i] = dp[i-1] + dp[i-2]$

3. **Base:** $dp[1] = 1, dp[2] = 2$

4. **Answer:** $dp[n]$ — আর শুধু দুইটা ভেরিয়েবল রাখলেই হয়

PHP

```
function climbStairs(int $n): int {
    if ($n <= 2) return $n;
    $oneBack = 2; // dp[i-1] (শুরুতে dp[2])
    $twoBack = 1; // dp[i-2] (শুরুতে dp[1])
    for ($i = 3; $i <= $n; $i++) {
        $curr = $oneBack + $twoBack;
        $twoBack = $oneBack;
        $oneBack = $curr;
    }
    return $oneBack;
}
echo climbStairs(3) . " " . climbStairs(5) . " " . climbStairs(45); // 3 8 1836311903
```

Time: $O(n)$

Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ i	1	2	3	4	5
dp[i]	1	2	$1+2 = 3$	$2+3 = 5$	$3+5 = 8$

৩ নম্বর ধাপে পৌঁছানোর ৩টা উপায়: $1+1+1$, $1+2$, $2+1$ — হাতে গুনে মিলিয়ে নাও।

এজ কেস (এগুলো ভুলবে না)

- $n = 1 \rightarrow 1$
- $n = 2 \rightarrow 2$

ফলো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"১, ২ বা ৩ ধাপ উঠতে পারলে?" — $dp[i] = dp[i-1] + dp[i-2] + dp[i-3]$ । "প্রতিটা ধাপের খরচ থাকলে সর্বনিম্ন খরচ?" — LeetCode 746 (প্র্যাকটিসে আছে)।

প্রশ্ন 5.4

House Robber

পাশাপাশি বাড়ি বাদ দিয়ে সর্বোচ্চ টাকা

Medium

LeetCode 198

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

"নেব না নেব না" (take or skip) সিদ্ধান্তের DP। এই প্যাটার্ন অনেক DP প্রশ্নের ভিত্তি।

সমস্যাটা কী?

একটা রাস্তায় বাড়িগুলোতে `nums[i]` টাকা আছে। পাশাপাশি দুইটা বাড়িতে চুরি করলে অ্যালার্ম বাজবে। অ্যালার্ম না বাজিয়ে সর্বোচ্চ কত টাকা নেওয়া যায়?

উদাহরণ

Input: [1, 2, 3, 1]
 Output: 4 // বাড়ি 0 (1) + বাড়ি 2 (3)
 Input: [2, 7, 9, 3, 1]
 Output: 12 // 2 + 9 + 1

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

প্রতিটা বাড়ির সামনে দাঁড়িয়ে দুইটা পথ: (ক) এই বাড়িতে ঢুকি — তাহলে আগের বাড়িতে ঢোকা যাবে না, তাই "দুই বাড়ি আগ পর্যন্ত সেরা + এই বাড়ির টাকা"; (খ) এটা বাদ দিই — তাহলে "আগের বাড়ি পর্যন্ত সেরা"। দুটোর মধ্যে যেটা বেশি।

পদ্ধতি ১: Recursion — প্রতিটা বাড়িতে নেব/নেব না

$rob(i) = \max(rob(i-1), rob(i-2) + nums[i])$ । প্রতিটা বাড়িতে দুইটা শাখা $\rightarrow O(2^n)$ ।

PHP

```
function robRec(array $nums, int $i): int {
    if ($i < 0) return 0;
    return max(robRec($nums, $i - 1),           // এই বাড়ি বাদ
               robRec($nums, $i - 2) + $nums[$i]); // এই বাড়ি নিলাম
}
$h = [2, 7, 9, 3, 1];
echo robRec($h, count($h) - 1); // 12
```

Time: $O(2^n)$ Space: $O(n)$

পদ্ধতি ২: DP (Bottom-up, $O(1)$ space)

- State:** $dp[i]$ = প্রথম $i+1$ টা বাড়ি থেকে সর্বোচ্চ টাকা
- Transition:** $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$
- Base:** $dp[0] = nums[0]$, $dp[1] = \max(nums[0], nums[1])$

4. আবার শুধু দুইটা ভেরিয়েবল যথেষ্ট

PHP

```
function rob(array $nums): int {
    $prev2 = 0; // dp[i-2]
    $prev1 = 0; // dp[i-1]
    foreach ($nums as $money) {
        $curr = max($prev1, $prev2 + $money); // বাদ দিই, নাকি নিই?
        $prev2 = $prev1;
        $prev1 = $curr;
    }
    return $prev1;
}
echo rob([1, 2, 3, 1]) . " " . rob([2, 7, 9, 3, 1]); // 4 12
```

Time: O(n)

Space: O(1)

ড্রাই রান — ধাপে ধাপে দেখি

বাড়ি i	টাকা	বাদ দিলে (dp[i-1])	নিলে (dp[i-2] + টাকা)	dp[i] = max
0	2	0	0 + 2 = 2	2
1	7	2	0 + 7 = 7	7
2	9	7	2 + 9 = 11	11
3	3	11	7 + 3 = 10	11
4	1	11	11 + 1 = 12	12

সবুজ বাড়িগুলো নেওয়া হয়েছে: 2 + 9 + 1 = 12

2	7	9	3	1
0	1	2	3	4

এজ কেস (এগুলো ভুলবে না)

- খালি array → 0
- এক বাড়ি → সেই টাকা
- দুই বাড়ি → বড়টা

ইন্টারভিউতে যা বলবে

- একটা ভুল ধারণা ভাঙো: "জোড় index বনাম বিজোড় index-এর যোগফল" সবসময় ঠিক না — [2,1,1,2]-এ উত্তর 4 (প্রথম আর শেষ), যা দুটো দলেই পড়ে না।

অধ্যায় ৩: Greedy Algorithm

Greedy মানে প্রতিটা ধাপে **এই মুহূর্তের সেরা** সিদ্ধান্ত নেওয়া, ভবিষ্যৎ নিয়ে না ভেবে — আর আশা করা যে এভাবে শেষে সেরা উত্তর আসবে। সহজ আর দ্রুত, কিন্তু **সবসময় সঠিক না**।

বাস্তব উদাহরণ: দোকানদারের খুচরা

৭৩ টাকা ফেরত দিতে হবে, নোট আছে ৫০, ২০, ১০, ৫, ২, ১। দোকানদার সবসময় সবচেয়ে বড় নোট আগে দেয়: ৫০ + ২০ + ২ + ১ = ৪টা নোট। বাংলাদেশি টাকার নোটে এই greedy সবসময় সবচেয়ে কম নোট দেয়। কিন্তু অদ্ভুত নোট হলে (১, ৩, ৪) ব্যর্থ হয় — পরের প্রশ্নে দেখবো।

	Greedy	Dynamic Programming
সিদ্ধান্ত	এখনকার সেরা, আর ফিরে দেখে না	সব সম্ভাবনা বিবেচনা করে (ছোট সমস্যার উত্তর ব্যবহার করে)
গতি	সাধারণত দ্রুত (প্রায়ই sort + এক পাস)	তুলনামূলক ধীর, বেশি মেমোরি
সঠিকতা	শুধু নির্দিষ্ট সমস্যায় (প্রমাণ লাগে)	সবসময় সঠিক (যদি DP প্রযোজ্য হয়)
উদাহরণ	Activity selection, Merge intervals, স্বাভাবিক নোটে খুচরা	Coin change (যেকোনো কয়েন), Knapsack

Greedy কখন কাজ করে?

যখন "এখনকার সেরা পছন্দ" কখনো ভবিষ্যতের সেরা উত্তর নষ্ট করে না। ক্লাসিক উদাহরণ — **Activity Selection**: সবচেয়ে বেশি মিটিং ধরাতে হলে যে মিটিং **সবচেয়ে আগে শেষ হয়** সেটা বেছে নাও, তারপর তার পরে শুরু হওয়া মিটিংগুলোর মধ্যে আবার সবচেয়ে আগে যেটা শেষ হয়। ইন্টারভিউতে greedy সলিউশন দিলে একটা ছোট counter-example খুঁজে দেখো কাজ করে কি না।

প্রশ্ন 5.5

Coin Change

সবচেয়ে কম কয়েনে টাকা বানাও — Greedy বনাম DP

Medium

LeetCode 322

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Greedy কেন সবসময় কাজ করে না তার সবচেয়ে বিখ্যাত উদাহরণ। "Greedy আর DP-র পার্থক্য কী?" প্রশ্নের নিখুঁত উত্তর এটা।

সমস্যাটা কী?

`coins` (প্রতিটা কয়েন অসীম সংখ্যায় আছে) আর `amount` দেওয়া আছে। `amount` বানাতে সবচেয়ে কম কয়টা কয়েন লাগবে? বানানো না গেলে -1।

উদাহরণ

```
Input: coins = [1, 2, 5], amount = 11
Output: 3      // 5 + 5 + 1
Input: coins = [1, 3, 4], amount = 6
Output: 2      // 3 + 3 (greedy দেয় 4+1+1 = 3টা!)
Input: coins = [2], amount = 3
Output: -1
```

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

৬ টাকা বানাতে হবে, কয়েন ১, ৩, ৪। লোভী মানুষ আগে সবচেয়ে বড় ৪ নেয়, বাকি ২ = ১ + ১ → ৩টা কয়েন। কিন্তু বুদ্ধিমান মানুষ দেখে ৩ + ৩ = ২টা কয়েন! এখনকার "সেরা" (৪) আসলে ভুল পছন্দ ছিল।

পদ্ধতি ১: Greedy — ভুল হতে পারে!

কয়েন বড় থেকে ছোট সাজাও, প্রতিবার যত বেশি সম্ভব বড় কয়েন নাও।

PHP

```
function coinChangeGreedy(array $coins, int $amount): int {
    rsort($coins); // বড় থেকে ছোট
    $count = 0;
    foreach ($coins as $c) {
        $count += intval($amount / $c); // এই কয়েন যতগুলো সম্ভব
        $amount %= $c;
    }
    return $amount === 0 ? $count : -1;
}

echo coinChangeGreedy([1, 2, 5], 11); // 3 ✓ ঠিক
echo " " . coinChangeGreedy([1, 3, 4], 6); // 3 ✗ ভুল! আসল উত্তর 2
```

Time: $O(k \log k)$ Space: $O(1)$

[1, 3, 4]-এ ভুল উত্তর দেয় — তাই এই প্রস্নে greedy গ্রহণযোগ্য না।

পদ্ধতি ২: DP (Bottom-up) — সবসময় সঠিক

1. **State:** $dp[x]$ = x টাকা বানাতে সর্বনিম্ন কয়েন
2. **Transition:** প্রতিটা কয়েন c-এর জন্য: $dp[x] = \min(dp[x], dp[x - c] + 1)$
3. **Base:** $dp[0] = 0$, বাকি সব "অসীম"
4. **Answer:** $dp[amount]$ (অসীম থাকলে -1)

PHP

```
function coinChange(array $coins, int $amount): int {
    $INF = $amount + 1; // "অসীম" (এর বেশি কয়েন কখনো লাগবে না)
    $dp = array_fill(0, $amount + 1, $INF);
    $dp[0] = 0;
    for ($x = 1; $x <= $amount; $x++) {
        foreach ($coins as $c) {
            if ($c <= $x) {
                $dp[$x] = min($dp[$x], $dp[$x - $c] + 1); // c কয়েনটা শেষে নিলে
            }
        }
    }
    return $dp[$amount] > $amount ? -1 : $dp[$amount];
}

echo coinChange([1, 2, 5], 11); // 3
echo " " . coinChange([1, 3, 4], 6); // 2 ✓
echo " " . coinChange([2], 3); // -1
```

Time: $O(\text{amount} \times k)$

Space: $O(\text{amount})$

k = কয়েনের ধরন।

ড্রাই রান — ধাপে ধাপে দেখি

coins = [1, 3, 4], amount = 6:

x	c=1 থেকে	c=3 থেকে	c=4 থেকে	dp[x]
0	—	—	—	0
1	$dp[0] + 1 = 1$	—	—	1
2	$dp[1] + 1 = 2$	—	—	2
3	$dp[2] + 1 = 3$	$dp[0] + 1 = 1$	—	1
4	$dp[3] + 1 = 2$	$dp[1] + 1 = 2$	$dp[0] + 1 = 1$	1
5	$dp[4] + 1 = 2$	$dp[2] + 1 = 3$	$dp[1] + 1 = 2$	2
6	$dp[5] + 1 = 3$	$dp[3] + 1 = 2$	$dp[2] + 1 = 3$	2

dp টেবিল। $dp[6] = dp[3] + 1$ — মানে 3 টাকা (১ কয়েন) + আরেকটা 3

0	1	2	1	1	2	2
0	1	2	3	4	5	6

এজ কেস (এগুলো ভুলবে না)

- amount = 0 $\rightarrow$ 0
- কোনোভাবেই বানানো না গেলে $\rightarrow$ -1
- একটা কয়েন amount-এর চেয়ে বড় হলে সেটা বাদ

ইন্টারভিউতে যা বলবে

- বলো: "Greedy শুধু canonical coin system-এ (যেমন ১, ২, ৫, ১০...) কাজ করে। যেকোনো কয়েনের জন্য DP লাগে, কারণ প্রতিটা amount-এর জন্য সব কয়েন বিবেচনা করি।"

প্রশ্ন 5.6

Merge Intervals

ওভারল্যাপ করা সময়-সীমাগুলো জোড়া লাগাও

Medium

LeetCode 56

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Sort + Greedy-র সবচেয়ে জনপ্রিয় প্রশ্ন। বাস্তবে ক্যালেন্ডার, বুকিং সিস্টেম, শিফট প্ল্যানিং — সব জায়গায় এই যুক্তি লাগে।

সমস্যাটা কী?

Interval-এর একটা array দেওয়া আছে, প্রতিটা `[start, end]`। ওভারল্যাপ করা সব interval মিশিয়ে দাও আর ওভারল্যাপ-মুক্ত interval-এর list রিটার্ন করো।

উদাহরণ

Input: `[[1,3], [2,6], [8,10], [15,18]]`

Output: `[[1,6], [8,10], [15,18]]`

Input: `[[1,4], [4,5]]`

Output: `[[1,5]]` // শেষ আর শুরু ছুঁয়ে গেলে-ও মিশবে

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

একটা মিটিং রুমের দিনের বুকিং: ১টা-৩টা, ২টা-৬টা, ৮টা-১০টা। রুম আসলে কখন কখন ব্যস্ত? ১-৩ আর ২-৬ ওভারল্যাপ করে, তাই ১টা-৬টা ব্যস্ত। শুরুর সময় অনুযায়ী সাজিয়ে নিলে শুধু আগের ব্লকের সাথে মেলালেই হয়।

পদ্ধতি ১: বারবার সব জোড়া মেলানো

যতক্ষণ কোনো দুইটা interval ওভারল্যাপ করে, তাদের মিশিয়ে দাও। প্রতিবার সব জোড়া চেক করতে হয় → অনেক ধীর।

PHP

```
function mergeBrute(array $iv): array {
    $changed = true;
    while ($changed) {
        $changed = false;
        for ($i = 0; $i < count($iv) && !$changed; $i++) {
            for ($j = $i + 1; $j < count($iv); $j++) {
                if ($iv[$i][0] <= $iv[$j][1] && $iv[$j][0] <= $iv[$i][1]) { //
ওভারল্যাপ?
                    $iv[$i] = [min($iv[$i][0], $iv[$j][0]), max($iv[$i][1], $iv[$j]
[1])];
                    array_splice($iv, $j, 1);
                    $changed = true; break;
                }
            }
        }
        sort($iv);
        return $iv;
    }
}
echo json_encode(mergeBrute([[1, 3], [2, 6], [8, 10], [15, 18]]));
```

Time: $O(n^3)$ Space: $O(1)$

পদ্ধতি ২: Sort + এক পাস (Greedy)

1. start অনুযায়ী sort করো
2. প্রথম interval result-এ রাখো
3. পরের প্রতিটা: তার start $\leq$ result-এর শেষ interval-এর end হলে $\rightarrow$ ওভারল্যাপ, end = max(দুটোর end)
4. নইলে নতুন interval হিসেবে result-এ যোগ করো

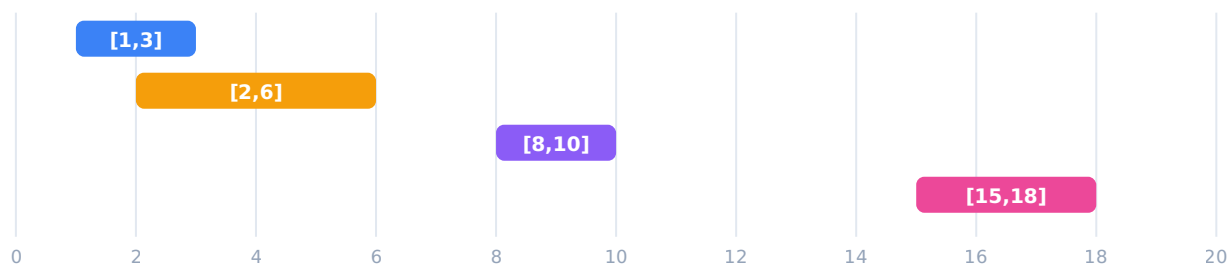
PHP

```
function merge(array $intervals): array {
    if (empty($intervals)) return [];
    usort($intervals, fn($a, $b) => $a[0] <=> $b[0]); // start অনুযায়ী sort
    $result = [$intervals[0]];
    for ($i = 1; $i < count($intervals); $i++) {
        [$start, $end] = $intervals[$i];
        $last = count($result) - 1;
        if ($start <= $result[$last][1]) { // ওভারল্যাপ
            $result[$last][1] = max($result[$last][1], $end);
        } else {
            $result[] = [$start, $end]; // নতুন ব্লক
        }
    }
    return $result;
}
echo json_encode(merge([[1, 3], [2, 6], [8, 10], [15, 18]])); // [[1,6],[8,10],
[15,18]]
echo " " . json_encode(merge([[1, 4], [4, 5]])); // [[1,5]]
```

Time: $O(n \log n)$ — sort-এর জন্যSpace: $O(n)$ — result

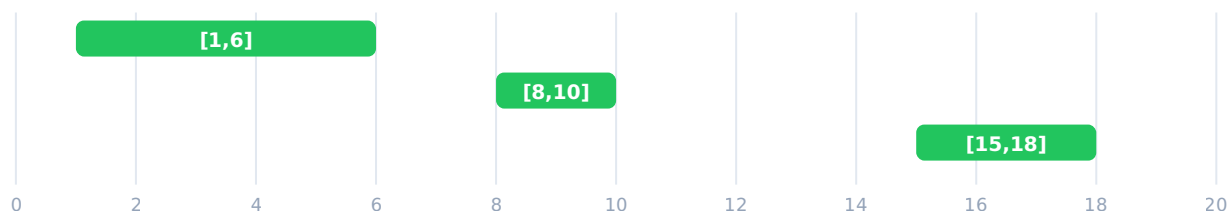
ড্রাই রান — ধাপে ধাপে দেখি

Sort করা interval-গুলো (সময় রেখায়):



interval	result-এর শেষ	ওভারল্যাপ?	result
[1,3]	—	—	[[1,3]]
[2,6]	[1,3]	$2 \leq 3 \rightarrow$ হ্যাঁ	[[1,6]]
[8,10]	[1,6]	$8 > 6 \rightarrow$ না	[[1,6], [8,10]]
[15,18]	[8,10]	$15 > 10 \rightarrow$ না	[[1,6], [8,10], [15,18]]

ফলাফল:



এজ কেস (এগুলো ভুলবে না)

- খালি input $\rightarrow []$
- একটা interval পুরোপুরি আরেকটার ভেতরে $[[1,10],[2,3]] \rightarrow \max()$ থাকায় end 10-ই থাকে
- ছুঁয়ে যাওয়া $[1,4],[4,5] \rightarrow$ মিশবে ($\leq$)

ইন্টারভিউতে যা বলবে

- PHP-তে 2D array sort: `usort($a, fn($x, $y) => $x[0] <=> $y[0]);` — spaceship operator মনে রাখো।
- কেন greedy কাজ করে: sort করার পর কোনো interval শুধু তার ঠিক আগের মিশ্রিত ব্লকের সাথেই ওভারল্যাপ করতে পারে।

থিওরি প্রশ্নোত্তর (ইন্টারভিউ স্টাইল)

প্রতিটা উত্তর ইন্টারভিউতে ৩০-৬০ সেকেন্ডে বলার মতো করে লেখা। মুখস্থ না করে নিজের ভাষায় বলার প্র্যাকটিস করো।

প্রশ্ন 1: Recursion কী? এর দুইটা অপরিহার্য অংশ কী?

ফাংশন নিজেকে ছোট ইনপুট দিয়ে কল করে সমস্যা সমাধান করে। অংশ দুটো: **base case** — যেখানে recursion থামে ও সরাসরি উত্তর দেয়; **recursive case** — সমস্যাকে ছোট করে নিজেকে কল করে। Base case না থাকলে বা সমস্যা ছোট না হলে অসীম recursion → stack overflow।

প্রশ্ন 2: Stack overflow কেন হয়?

প্রতিটা ফাংশন কল call stack-এ মেমোরি নেয়। Recursion খুব গভীরে গেলে (base case নেই বা ইনপুট বিশাল) stack-এর নির্দিষ্ট মেমোরি ফুরিয়ে যায়। সমাধান: base case ঠিক করা, বা recursion-কে loop (iterative) বানানো।

প্রশ্ন 3: Recursion বনাম Iteration — কোনটা ভালো?

Recursion কোডকে ছোট ও স্বাভাবিক করে (tree, divide & conquer), কিন্তু call stack-এর জন্য $O(\text{depth})$ মেমোরি লাগে আর function call-এর খরচ আছে। Iteration সাধারণত দ্রুত আর $O(1)$ stack। সমস্যার গঠন recursive হলে recursion, সহজ লুপে হলে iteration।

প্রশ্ন 4: Memoization কী?

Recursive ফাংশনের ফলাফল একটা cache-এ রেখে দেওয়া, যাতে একই ইনপুটে আবার হিসাব না করে cache থেকে দেওয়া যায়। এতে overlapping subproblem-ওয়ালা সমস্যা exponential থেকে polynomial হয়ে যায় — যেমন Fibonacci $O(2^n) \rightarrow O(n)$ ।

প্রশ্ন 5: Dynamic Programming কী? কখন ব্যবহার করবে?

বড় সমস্যাকে ছোট overlapping subproblem-এ ভেঙে, প্রতিটা একবার সমাধান করে সংরক্ষণ করা। দুই শর্ত: overlapping subproblems আর optimal substructure। সংকেত: "সর্বোচ্চ/সর্বনিম্ন", "কয়টা উপায়", "সম্ভব কি না" — আর প্রতিটা ধাপে পছন্দ আছে।

প্রশ্ন 6: Top-down আর Bottom-up DP-র পার্থক্য?

Top-down = recursion + memoization, বড় সমস্যা থেকে শুরু, শুধু দরকারি state হিসাব হয়, কিন্তু recursion-এর stack লাগে। Bottom-up = tabulation, ছোট থেকে টেবিল ভরে বড়তে যায়, loop দিয়ে, stack overflow ঝুঁকি নেই, প্রায়ই space optimize করা যায়।

প্রশ্ন 7: Greedy algorithm কী? DP থেকে পার্থক্য কী?

Greedy প্রতি ধাপে তখনকার সেরা পছন্দ নেয় আর কখনো ফিরে দেখে না — দ্রুত, কিন্তু সবসময় সঠিক না। DP সব সম্ভাব্য পছন্দ বিবেচনা করে ছোট সমস্যার উত্তর মিলিয়ে সেরাটা বের করে — ধীর, কিন্তু সঠিক। Coin change [1,3,4], amount 6: greedy 4+1+1 (৩টা), DP 3+3 (২টা)।

প্রশ্ন 8: Divide & Conquer আর DP-র পার্থক্য?

দুটোই সমস্যা ভাঙে। Divide & conquer-এ ভাগগুলো **আলাদা** (merge sort-এর দুই অর্ধেক কখনো একই কাজ করে না), তাই cache লাগে না। DP-তে ভাগগুলো **ওভারল্যাপ** করে (fib(3) বারবার), তাই সংরক্ষণ করে রাখি।

প্রশ্ন 9: একটা Greedy সমস্যার উদাহরণ দাও যেখানে greedy সঠিক।

Activity selection: সবচেয়ে বেশি মিটিং ধরাতে, মিটিংগুলো শেষ হওয়ার সময় অনুযায়ী sort করে সবচেয়ে আগে শেষ হওয়াটা নিই, তারপর যেগুলো তার পরে শুরু হয় তাদের মধ্যে আবার সবচেয়ে আগে শেষ হওয়াটা। আগে শেষ হওয়া মিটিং বাকিদের জন্য সবচেয়ে বেশি সময় রেখে দেয়, তাই এটা সবসময় সেরা।

প্র্যাকটিস প্রবলেম (নিজে সলভ করো)

আগে নিজে ২০ মিনিট চেষ্টা করো। আটকে গেলে হিন্ট দেখো। তারপরও না পারলে সলিউশন দেখে, পরদিন আবার নিজে লেখো।

1. Min Cost Climbing Stairs (Easy · LeetCode 746)

প্রতিটা ধাপের খরচ দেওয়া। উপরে উঠতে সর্বনিম্ন খরচ।

হিন্ট: $dp[i] = cost[i] + \min(dp[i-1], dp[i-2])$ । উত্তর $\min$ (শেষ দুইটা)।

2. Pow(x, n) (Medium · LeetCode 50)

x^n বের করো, n নেগেটিভও হতে পারে।

হিন্ট: আজকের powerFast! n নেগেটিভ হলে $1 / power(x, -n)$ ।

3. Unique Paths (Medium · LeetCode 62)

$m \times n$ গ্রিডে উপর-বাম থেকে নিচ-ডানে (শুধু ডানে/নিচে) কয়টা পথ?

হিন্ট: $dp[r][c] = dp[r-1][c] + dp[r][c-1]$ । প্রথম সারি আর কলাম সব 1।

4. Jump Game (Medium · LeetCode 55)

$nums[i]$ = সেখান থেকে সর্বোচ্চ কত লাফ। শেষে পৌঁছানো যায়?

হিন্ট: Greedy: সর্বোচ্চ কতদূর পৌঁছানো যায় ($maxReach$) রাখো; $i > maxReach$ হলে false।

5. Assign Cookies (Easy · LeetCode 455)

প্রতিটা বাচ্চার চাহিদা আর কুকির সাইজ। সর্বোচ্চ কতজনকে খুশি করা যায়?

হিন্ট: Greedy: দুটোই sort। ছোট কুকি দিয়ে সবচেয়ে কম চাহিদার বাচ্চাকে আগে খুশি করো।

6. Subsets (Medium · LeetCode 78)

$[1, 2, 3]$ -এর সব subset।

হিন্ট: Recursion: প্রতিটা element-এ দুই পথ — নাও বা নিও না। শেষে পৌঁছালে বর্তমান list result-এ যোগ করো।

চিট শিট — দিন ৫

Recursion টেমপ্লেট

PHP — টেমপ্লেট

```
function solve($input) {
    if (/* সবচেয়ে ছোট কেস */) return /* সরাসরি উত্তর */; // base case
    $smaller = solve(/* ছোট ইনপুট */); // recursive case
    return /* $smaller ব্যবহার করে এখনকার উত্তর */;
}
```

DP-র ৪ ধাপ

ধাপ	State	Transition	Base
Fibonacci	dp[i] = i-তম সংখ্যা	dp[i-1] + dp[i-2]	dp[0]=0, dp[1]=1
Climbing Stairs	dp[i] = i-তে যাওয়ার উপায়	dp[i-1] + dp[i-2]	dp[1]=1, dp[2]=2
House Robber	dp[i] = i পর্যন্ত সর্বোচ্চ	max(dp[i-1], dp[i-2] + nums[i])	dp[-1]=0
Coin Change	dp[x] = x-এর সর্বনিম্ন কয়েন	min(dp[x - c] + 1)	dp[0]=0

কোনটা কখন

প্রশ্নে যা দেখাবে	চিন্তা করো
একই ছোট সমস্যা বারবার	Memoization / DP
"কয়টা উপায়", "সর্বোচ্চ/সর্বনিম্ন", পছন্দ আছে	DP
Interval, schedule, "সবচেয়ে বেশি কয়টা"	Sort + Greedy
অধিক করে সমাধান	Divide & Conquer
সব সম্ভাবনা লাগবে (subset, permutation)	Recursion / Backtracking

আজকের নিজেকে যাচাই

- ☐ Fibonacci ৪ ভাবে লিখতে ও complexity বলতে পারি
- ☐ DP-র ৪ ধাপ (state, transition, base, answer) বলতে পারি
- ☐ Climbing Stairs আর House Robber নিজে লিখতে পারি
- ☐ Greedy কেন coin change-এ ব্যর্থ — উদাহরণ দিতে পারি
- ☐ Merge Intervals-এ sort কেন দরকার বলতে পারি